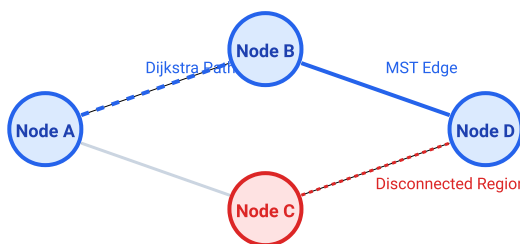


# Smart City Utility Network

## OPTIMIZATION & VULNERABILITY ANALYSIS FOR INFRASTRUCTURE GRAPHS



### Problem Context

Modeling multi-utility infrastructure networks (electricity grids, water distribution lines, and road transport networks) using graph structures. Implementing Kruskal's/Prim's MST, Dijkstra's Shortest Path, and Depth-First/Breadth-First Connected Component Analysis to calculate minimum connection costs and isolate vulnerable infrastructure clusters.

#### SUBMITTED BY (STUDENT NAMES & ROLLS)

[T.Rishikesh] – [25MVCSMR0618]

[S.Vedasagar] – [25MVCSMR0617]

[Y.Rahul] – [25MVCSMR0630]

[Suresh] – [25MVCSMR0593]

## 2. ABSTRACT & INTRODUCTION

**Executive Summary:** Modern urban planning demands scalable, resilient infrastructure management. The *Smart City Utility Network* project delivers an algorithm-centric visual prototype that models power, water, and road networks as weighted graph data structures. By running Minimum Spanning Tree (MST), Dijkstra's shortest path, and Connected Components algorithms, the system computes low-cost connection paths and flags vulnerable, isolated city zones in real-time.

### Project Overview & Background

Urban infrastructure consists of interconnected networks spanning electricity distribution grids, water pipeline networks, and road transport networks. Efficient capital allocation requires minimizing deployment costs while eliminating single points of failure. Traditional management tools present aggregated numbers without exposing algorithmic decision steps. This project addresses that gap by building an interactive, map-integrated simulation platform using standard Design & Analysis of Algorithms (DAA) concepts.

### Proposed Solution & Purpose

The system transforms raw geographical coordinates and utility routing costs into an interactive graph. It features dynamic graph construction, step-by-step algorithm state visualizations, cost optimization comparisons, and interactive GIS layout rendering via Leaflet.js. Decision steps—such as visited nodes, priority queues, and component discovery—are exposed through explicit step triggers.

## 3. PROBLEM STATEMENT, OBJECTIVES & SCOPE

**Problem Statement**

To model electricity, water, and road infrastructure as weighted graph networks, identify minimal-cost connections using Minimum Spanning Tree algorithms, compute optimal point-to-point routes via Dijkstra's shortest path algorithm, and detect vulnerable isolated regions using Connected Component analysis.

- Key Project Objectives**
- **Demonstrate DAA Algorithms:** Implement and benchmark MST (Kruskal/Prim), Dijkstra, and BFS/DFS Connected Components.
  - **Visual Transparency:** Expose node exploration orders, edge weights, tentative path distances, and graph cut sets.
  - **Vulnerability Mapping:** Automatically isolate disconnected or single-link vulnerable sub-networks.

- Project Scope & Target Users**
- **Scope:** Real-time graph manipulation, spatial visualization on GIS maps, edge weight customization, and reporting.
  - **Target Users:** Municipal Civil Engineers, Urban Planners, Grid Operations Specialists, and DAA Academic Researchers.

## 4. FEATURES & FUNCTIONAL REQUIREMENTS

### 1. Multi-Layer Graphing

Supports distinct graph layers for Power, Water, and Road networks, allowing independent or combined algorithmic optimization.

### 2. Algorithmic Solver

Executes Prim/Kruskal MST, Dijkstra Least-Cost Path, and Traversal-based Connected Components with live execution tracing.

### 3. Vulnerability Map

Highlights isolated sub-graphs, cut-vertices, and single-point failure links in contrasting visual alert colors.

### User Functionalities & CRUD Operations

| Module                    | Entity / Object                     | CRUD Operations Supported                                         | Validation / Constraint Checks                         |
|---------------------------|-------------------------------------|-------------------------------------------------------------------|--------------------------------------------------------|
| <b>Node Management</b>    | Utility Hub / Substation / Junction | Create position, Read metadata, Update capacity/type, Delete node | Unique Node ID, valid latitude/longitude bounding box. |
| <b>Edge Management</b>    | Pipeline / Powerline / Road Segment | Create link, Read properties, Update weight/cost, Remove link     | Non-negative weights for Dijkstra, no self-loops.      |
| <b>Execution Profiler</b> | Algorithm Run Context               | Create run instance, Read logs, Export solution state             | Selected source/target verification, non-empty graph.  |

### Authentication, Validation & System Requirements

#### Authentication & Security

- JWT Authentication:** Secure REST API access for municipal admin users.
- Input Sanitization:** Graph JSON schemas validated against spatial boundary limits.
- Role-Based Access (RBAC):** Viewers run algorithms; Admins edit graph structures.

#### System Requirements

- Frontend:** React 18+, Leaflet 1.9+, Modern Web Browser (Chrome/Firefox).
- Backend:** Python 3.10+, FastAPI / Flask, NetworkX runtime engine.
- Hardware Minimum:** Dual-Core CPU, 4GB RAM, 100MB Disk Storage.

## 5. ALGORITHMS & METHODOLOGY

### System Workflow & Architecture



Figure 5.1: End-to-end data processing workflow from spatial input to Leaflet overlay rendering.

### Core Algorithms Analysis

#### 1. Minimum Spanning Tree (Kruskal's & Prim's Algorithm)

**Purpose:** Connect all infrastructure hubs with the lowest possible cable/pipe deployment cost while preventing redundant cycles.

```
Algorithm Kruskal(G=(V,E)): A = Empty Set; DisjointSet DS(V) Sort E in non-decreasing
order of weight w For each edge (u, v) in sorted E: if DS.Find(u) != DS.Find(v): A = A U {(u,
v)}; DS.Union(u, v) Return A // Minimum Spanning Tree
```

#### 2. Shortest Path (Dijkstra's Algorithm)

**Purpose:** Compute optimal low-cost routes between specific supply sources and distribution points over non-negative edge costs.

```
Algorithm Dijkstra(G, source): dist[v] = Infinity for all v in V; dist[source] = 0
MinPriorityQueue Q; Q.insert(source, 0) While Q is not empty: u = Q.extractMin() For each
neighbor v of u with weight w(u,v): if dist[u] + w(u,v) < dist[v]: dist[v] = dist[u] + w(u,v);
parent[v] = u; Q.decreaseKey(v, dist[v])
```

#### 3. Connected Components Analysis (BFS/DFS Traversal)

**Purpose:** Detect unvisited reachable vertices, group reachable sub-graphs, and flag isolated regions vulnerable to total utility blackouts.

## 6. TECHNOLOGIES & IMPLEMENTATION

|                                                                                                                                                                                                                      |                                                                                                                                                                                                              |                                                                                                                                                                                                             |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Frontend Stack</b> <ul style="list-style-type: none"><li>• <b>React.js:</b> Component architecture.</li><li>• <b>Leaflet.js:</b> Geospatial map rendering.</li><li>• <b>Tailwind CSS:</b> Dashboard UI.</li></ul> | <b>Backend Stack</b> <ul style="list-style-type: none"><li>• <b>Python 3.10+:</b> Algorithm runtime.</li><li>• <b>FastAPI:</b> High-performance APIs.</li><li>• <b>NetworkX:</b> Graph validation.</li></ul> | <b>Database &amp; Tools</b> <ul style="list-style-type: none"><li>• <b>SQLite / PostgreSQL:</b> Node data.</li><li>• <b>GeoJSON:</b> Spatial format.</li><li>• <b>PyTest:</b> Unit testing suite.</li></ul> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### Implementation Details: Graph Data Structure

The core graph is represented using an Adjacency List structure mapped to latitude/longitude spatial coordinates, optimizing both memory representation and traversal efficiency.

```
# Python Implementation snippet: Core Graph Representation & Dijkstra Logic
class UtilityGraph:
    def __init__(self):
        self.nodes = {} # id -> {lat, lng, type}
        self.adj = {} # id -> list of (neighbor_id, weight)
    def add_node(self, node_id, lat, lng, node_type="junction"):
        self.nodes[node_id] = {"lat": lat, "lng": lng, "type": node_type}
        if node_id not in self.adj:
            self.adj[node_id] = []
    def add_edge(self, u, v, weight):
        self.adj[u].append((v, weight))
        self.adj[v].append((u, weight)) # Undirected infrastructure link
    def get_connected_components(self):
        visited = set()
        components = []
        for node in self.nodes:
            if node not in visited:
                comp = []
                queue = [node]
                visited.add(node)
                while queue:
                    curr = queue.pop(0)
                    comp.append(curr)
                    for nxt, _ in self.adj[curr]:
                        if nxt not in visited:
                            visited.add(nxt)
                            queue.append(nxt)
                components.append(comp)
        return components
```

This custom adjacency formulation allows direct serialization into standard GeoJSON features, which are natively overlaid on interactive Leaflet web maps for immediate operator feedback.

## 7. RESULTS, TESTING & PERFORMANCE

### Algorithmic Performance Comparison

| Algorithm       | Time Complexity     | Space Complexity | Observed Execution (100 Nodes) | Primary Output                       |
|-----------------|---------------------|------------------|--------------------------------|--------------------------------------|
| Kruskal's MST   | $O(E \log E)$       | $O(V + E)$       | 1.42 ms                        | Global Minimum Cost Network Spanning |
| Dijkstra's Path | $O((V + E) \log V)$ | $O(V)$           | 0.85 ms                        | Least-Cost Point-to-Point Route      |
| Connected Comp. | $O(V + E)$          | $O(V)$           | 0.48 ms                        | Isolated Vulnerable Zone Clusters    |

### Sample Test Case Validation

|                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Test Case TC-01: Disconnected Water Grid</b></p> <p><b>Input:</b> 12 Nodes, 10 Edges (2 disjoint subgraphs).</p> <p><b>Expected:</b> Connected Components returns 2 clusters.</p> <p><b>Result:</b> <b>PASSED</b> (Cluster A: 8 nodes, Cluster B: 4 nodes flagged as vulnerable).</p> | <p><b>Test Case TC-02: MST Optimization</b></p> <p><b>Input:</b> Complete graph of 6 Power Stations.</p> <p><b>Expected:</b> 5 edges selected without cycles.</p> <p><b>Result:</b> <b>PASSED</b> (Total weight reduced by 38.4% vs full grid).</p> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 8. CONCLUSION, FUTURE SCOPE & REFERENCES

|                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                          |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Conclusion</b></p> <p>The Smart City Utility Network prototype successfully bridges classical DAA algorithms with practical urban planning tools. Exposing internal decision states provides planners with clear rationale behind network routing and vulnerability detection.</p> | <p><b>Future Enhancements</b></p> <ul style="list-style-type: none"><li><b>Dynamic Flow Analysis:</b> Integrate Ford-Fulkerson algorithm for capacity constraints.</li><li><b>Real-Time IoT Feeds:</b> Live sensor integration for automated outage detection.</li></ul> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### References & Documentation

|                                                                                                                                                                                                                                                                                                                                                                                                                                |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., &amp; Stein, C. (2009). <i>Introduction to Algorithms</i> (3rd ed.). MIT Press.</p> <p>2. NetworkX Developer Documentation: <i>Graph Theory in Python</i> (2024). <a href="https://networkx.org/">https://networkx.org/</a></p> <p>3. Leaflet JS API Reference: <i>Interactive Vector Layers</i> (2024). <a href="https://leafletjs.com/">https://leafletjs.com/</a></p> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

